STEP-BY-STEP WORKBOOK / NUMERICAL ML

softmax와 cross-entropy의 수치 버그

"A Fix for Softmax and Cross-Entropy" (Barron, Gnanapragasam, Tschannen · Google DeepMind · arXiv:2610.31642) 를
고3 수학 + ViT를 아는 분을 위해 한 걸음씩 풀어 쓴 안내서

- 왜 정답을 100% 확신하는 순간 학습이 오히려 망가지는지 설명할 수 있다.
- 부동소수점의 눈금만으로 그 고장 지점(정밀도 절벽)을 계산할 수 있다.
- 일곱 줄짜리 수정이 무엇을 바꾸는지, 내 프로젝트에 언제 적용할지 판단할 수 있다.

2026-10-24 · A4 인쇄용 · 11 STEP + 연습문제 10 + 정답

원문: arXiv:2610.31642 (CC BY 4.0) · 이 워크북은 같은 조건으로 배포
수치는 모두 로컬 계산 · 코드는 실행 검증 · design-craft 검토 v2

이 워크북을 읽는 법

이미 아는 것 위에 한 층만 얹습니다

이 워크북은 여러분이 **ViT의 attention**을 안다고 가정합니다. 그러면 `softmax`는 이미 아는 단어입니다. attention에서 Q·K를 곱해 점수를 만들고 softmax로 확률처럼 바꾸죠. 이 워크북이 하는 일은 그 softmax를 **손실함수 쪽에서** 다시 보는 것입니다. 수학은 고3 수준(지수·로그·시그마·간단한 미분)을 넘지 않습니다.

여러분이 이미 아는 것	이 워크북이 붙이는 것
softmax는 점수를 확률로 바꾼다	그 확률이 **1에 너무 가까워지면** 무슨 일이 생기는가
cross-entropy는 정답 확률에 -log를 씌운다	그 손실의 **미분이 정확히** $q - p$ 라는 사실
float32, bf16은 "정밀도가 다르다"	정밀도가 **몇 비트**인지로 고장 지점을 **계산**하는 법
Adam은 gradient를 정규화한다	그래서 **0과 아주 작은 값**이 전혀 다르게 취급되는 이유
증류는 teacher의 확률을 따라간다	학생이 이미 teacher와 같을 때 **0이 아니게 되는** gradient

> 이 워크북의 약속
> ① 기호가 나오면 **바로 옆에 한국어 뜻**을 붙입니다. ② 행렬은 쓰지 않고 **숫자 세 개**로만 계산합니다. ③ 모든 숫자는 실제로 계산한 값입니다(손으로 어림한 값이 아닙니다).

11 스텝 지도

1. softmax는 비교를 확률로 바꾸는 기계
2. margin — 압승의 정도를 재는 자
3. 손실의 미분은 정확히 $q - p$
4. float는 눈금이다
5. 버그 1 — 분모가 1이 되는 순간
6. 버그 1의 결과 — 합이 0이 아니다
7. Adam은 0을 특별히 다룬다 (Slingshot)
8. 수정 1 — 1을 더하지 않는다
9. 버그 2 — 증류의 상쇄
10. 수정 2 — expm1과 log1p
11. 왜 기본값 $\epsilon = 10^{-8}$이 최악인가

> 이 논문을 한 문장으로
> "정답을 100% 확신하는 모델의 **정답 쪽 gradient가 정확히 0으로 반올림**되어, gradient 합이 0이라는 규칙이 깨진다. 그러면 Adam이 그 깨진 정보를 **증폭**해서 학습이 주기적으로 폭발한다."

STEP 01

softmax는 "비교"를 확률로 바꾸는 기계

> **왜 필요한가.** 버그를 이해하려면 softmax가 **무엇을 계산하는지**가 아니라 **어떤 형태로 계산하는지**를 봐야 합니다. 이 워크북의 나머지가 전부 이 형태 하나에 걸려 있습니다.

비유

세 명의 시험 점수 90, 80, 70점을 "누가 1등일 확률"로 바꾼다고 합시다. 점수를 그대로 비율로 쓰면 90/240 = 0.375인데, 이러면 1등도 37%밖에 안 됩니다. **차이를 증폭**시키고 싶으니 지수함수를 씁니다. e^{90}은 e^{80}보다 22,000배 큽니다. 그러면 1등 확률이 거의 1이 됩니다. **점수를 벌리는 정도를 조절하는 것이 지수**입니다.

수식

점수 z_k(이름: **로짓**, logit)를 확률 q_k로 바꿉니다.

$$q_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

기호	읽는 법	느낌
z_k	제트 케이	모델이 매긴 원점수. 로그 눈금이라 **차이**만 의미가 있음
q_k	큐 케이	k번 클래스일 확률. 전부 더하면 정확히 1
$\sum_j$	시그마 제이	모든 클래스에 대해 더하기

아주 작은 숫자 예

로짓이 $z = [2,\ 1,\ 0]$일 때 (클래스 3개):

k	z_k	e^{z_k}	q_k
0	2	7.389	0.665
1	1	2.718	0.245
2	0	1.000	0.090
합	–	11.107	1.000

로짓 차이는 1인데 확률 차이는 0.42입니다. **지수가 차이를 벌립니다.**

확인 문제 1-1

$z = [0, 0, 0]$이면 q는 얼마인가?

STEP 02

margin — "얼마나 이겼나"를 재는 자

> **왜 필요한가.** 확률 $q_t = 0.9999$와 0.99999는 표시해도 구분이 안 됩니다. 그래서 이 논문은 확률 대신 **margin**이라는 하나의 숫자로 모든 것을 설명합니다. 앞으로 나오는 절벽·평형·Slingshot이 전부 이 숫자 하나에 대한 이야기입니다.

비유

100m 달리기 1등이 2등보다 몇 초 앞섰는가. 0.01초면 사진 판독, 10초면 압승입니다. **확률(우승 확률)은 둘 다 0.999로 보이지만 margin은 0.01과 10으로 다릅니다.** margin은 "압승의 정도"를 재는 자입니다.

수식

정답 클래스를 t라 하면

$$m = z_t - \log \sum_{j \neq t} e^{z_j}, \qquad q_t = \frac{1}{1 + e^{-m}}$$

기호	읽는 법	느낌
m	엠 / 로짓 마진	정답이 "나머지 전부"보다 얼마나 큰가 (로그 눈금)
$\log \sum_{j \neq t} e^{z_j}$	로그-섬-이엑스피	오답들을 하나의 경쟁자로 합친 점수
$1/(1+e^{-m})$	시그모이드	margin을 확률로 되돌리는 함수

두 번째 식이 핵심입니다. **정답 확률은 margin 하나로 결정됩니다.**

아주 작은 숫자 예

margin m	정답 확률 q_t	상황
0	0.500	오답과 완전히 동점
5	0.993	꽤 확신
10	0.99995	매우 확신
17	0.99999996	확률로는 1과 구분 불가 (여기가 이 논문의 무대)

$m = 17$에서 $1 - q_t = 4.1 \times 10^{-8}$. **확률을 32비트로 저장하면 이 차이는 사라집니다.** STEP 4에서 그 이유를 봅니다.

확인 문제 2-1

$z_t = 5$이고 나머지 두 로짓이 $0, 0$일 때 margin은? (힌트: $\log(e^0 + e^0) = \ln 2 = 0.693$)

STEP 03

손실의 미분은 정확히 $q - p$

> **왜 필요한가.** 이 논문 전체가 "gradient가 이상하다"는 주장입니다. 그런데 이상하다는 것을 알려면 **정상이 무엇인지**를 알아야 합니다. 정상은 아주 단순합니다 — gradient는 확률에서 정답표를 뺀 것.

비유

오답 노트를 채점한다고 합시다. 정답 확률이 1에 못 미친 만큼(**음수**) 정답 쪽을 밀어 올리고, 오답마다 그 확률만큼(**양수**) 오답을 내립니다. **밀어 올린 총량 = 내린 총량**이라 합은 항상 0입니다. 이 "합이 0"이 예산 제약이고, 이 논문의 버그는 **이 제약이 깨지는 것**입니다.

수식

정답이 하나인 경우(one-hot) 손실은 $\mathcal{L} = -\log q_t = -z_t + \log \sum_j e^{z_j}$. 미분하면

$$\frac{\partial \mathcal{L}}{\partial z_k} = q_k - \mathbf{1}[k = t] \quad \Longrightarrow \quad \sum_k \frac{\partial \mathcal{L}}{\partial z_k} = 0$$

기호	읽는 법	느낌
$\mathcal{L}$	엘 / 손실	작을수록 좋은 점수. 정답 확률에 -log
$\partial \mathcal{L} / \partial z_k$	편미분	"z_k를 1 올리면 손실이 얼마나 변하나"
$\mathbf{1}[k = t]$	정답표	정답 칸만 1, 나머지 0

$-\log q_t$를 z_t로 미분하면 $q_t - 1$, 다른 칸으로 미분하면 q_k가 됩니다. **정답 칸만 음수, 나머지는 양수, 합은 0.**

아주 작은 숫자 예

STEP 1의 $z = [2, 1, 0]$, 정답 $t = 0$, $q = [0.665, 0.245, 0.090]$:

k	q_k	정답표	gradient $q_k - 1$	뜻
0 (정답)	0.665	1	−0.335	정답을 위로 밀어라
1	0.245	0	+0.245	이 오답을 아래로
2	0.090	0	+0.090	이 오답을 아래로
합	1.000	–	0.000	예산 균형

> 외워 둘 한 줄
> **gradient의 합은 언제나 0.** 이 워크북에서 벌어지는 모든 사고는 이 한 줄이 깨지는 사건입니다.

> 확인 문제 3-1
> $z = [0, 0]$ (클래스 2개), 정답 $t = 0$일 때 gradient를 구하고 합이 0인지 확인하라.

STEP 04

float는 눈금이다 — 그리고 눈금에는 한계가 있다

> **왜 필요한가.** 버그의 원인은 수학이 아니라 **저장 방식**입니다. "정밀도가 낮다"는 말을 **숫자 하나**로 바꿔야 STEP 5의 절벽을 계산할 수 있습니다.

비유

1m짜리 자에 1mm 눈금만 있으면 0.4mm는 잴 수 없고 **0mm로 반올림**됩니다. 컴퓨터의 실수도 같습니다 — **1 근처에서는 눈금 간격이 1.19×10^{-7}**(fp32)이고 그보다 작은 변화는 "없는 일"이 됩니다. 0.0001 근처에서는 훨씬 촘촘합니다. **같은 실수인데 위치에 따라 정밀도가 다릅니다.**

수식

실수는 $\pm 1.\text{mantissa} \times 2^{e}$ 꼴로 저장됩니다. 유효숫자가 p비트면 1 근처의 눈금 간격(ulp)은

$$\text{ulp}(1) = 2^{\,1-p}, \qquad \text{반올림 경계} = \frac{\text{ulp}}{2} = 2^{-p}$$

기호	읽는 법	느낌
p	피 / 유효 비트	가수부 비트 수. bf16=8, fp32=24, fp64=53
ulp	울프	그 위치에서 표현 가능한 **이웃한 두 수의 간격**
2^{-p}	반올림 경계	이보다 작은 변화는 더했을 때 **사라진다**

아주 작은 숫자 예

형식	p	1 근처 ulp	반올림 경계 2^{-p}
bf16	8	7.8×10^{-3}	3.9×10^{-3}
fp32	24	1.19×10^{-7}	5.96×10^{-8}
fp64	53	2.2×10^{-16}	1.1×10^{-16}

실측(fp32): `1.0 + 1e-7 = 1.0000001`, `1.0 + 3e-8 = 1.0` — **반올림 경계보다 작으면 흔적이 없다.**

> 주의
> "fp32니까 7자리는 맞다"는 말은 **1 근처에서만** 참입니다. 이 비대칭이 이 논문의 전부입니다.

> 확인 문제 4-1
> fp32에서 `1.0 + 3e-8`의 결과는? 이유도 한 줄로.

STEP 05

버그 1 — 분모가 정확히 1이 되는 순간

> **왜 필요한가.** 여기가 이 논문의 첫 번째이자 핵심 발견입니다. 표준 구현이 분모를 만드는 방식이 **정확히 1에 작은 수를 더하는** 형태라서, STEP 4의 눈금 한계에 그대로 걸립니다.

비유

0.001g까지 재는 저울이 있습니다. 여기에 0.00004g짜리 깃털을 올리면 **0g**으로 표시됩니다. 깃털이 없는 게 아니라 **저울이 못 읽는 것**입니다. 표준 구현의 분모가 바로 이 저울입니다.

수식

표준 구현은 로짓에서 최댓값을 빼서 $\tilde{z}_t = 0$으로 만듭니다. 그러면 분모는

$$\sum_i e^{\tilde{z}_i} = 1 + \underbrace{\sum_{j \neq t} e^{z_j - z_t}}_{e^{-m}} = 1 + e^{-m}$$

e^{-m}이 반올림 경계 2^{-p}보다 작아지면 $1 + e^{-m}$은 **정확히 1.0**이 됩니다. 그 지점이

$$e^{-m} < 2^{-p} \iff m > c_d \equiv p \ln 2$$

기호	읽는 법	느낌
c_d	씨 디 / 정밀도 절벽	이 margin을 넘으면 계산이 무너지는 **경계선**
$p \ln 2$	피 엘엔 투	비트 수 × 0.693. 외울 필요 없이 이 곱만 하면 됨

아주 작은 숫자 예 — 계산해서 확인

형식	$p \ln 2$	실제로 1.00이 되는 m
bf16	5.55	m ≳ 6
fp32	16.64	m = 17 → `1+e^-17 = 1.0` (실측)
fp64	36.74	m ≳ 37

논문의 값(bf16 5.5, fp32 16.6, fp64 36.7)과 일치합니다. **공식 하나로 세 정밀도를 모두 예측**합니다.

fp32 의 수직선, 1.0 근처를 확대

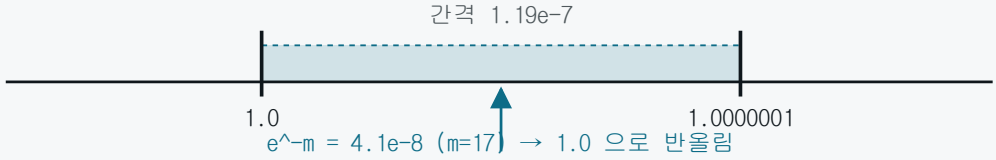


그림 1. 이 구간에 떨어지는 수는 더해도 1.0으로 반올림된다. 폭 = 반올림 경계 2^{-p}.

확인 문제 5-1

bf16($p = 8$)에서 절벽은 m 몇인가? 계산식과 값을 쓰라.

STEP 06

버그 1의 결과 — 오답 gradient만 살아남는다

> **왜 필요한가.** 분모가 1이 되는 것 자체는 사소해 보입니다. 문제는 **정답 쪽만** 계산이 망가지고 오답 쪽은 멀쩡하다는 비대칭입니다. STEP 3에서 배운 "합이 0"이 여기서 깨집니다.

비유

회의에서 찬성표는 큰 글씨라 읽히고 반대표는 작은 글씨라 안 읽힌다면, 표는 안 맞는데 결론은 나옵니다. **한쪽만 세는 회의**가 정확히 이 상황입니다.

수식

정답 칸은 $q_t = 1/(1+e^{-m}) = 1$이 되어 $g_t = q_t - 1 = 0$. 오답 칸은 1을 더하지 않고 $e^{z_j - z_t}$를 작은 수 근처에서 직접 계산하므로 살아 있습니다.

$$g_t = 0, \qquad g_j = \frac{e^{z_j - z_t}}{1 + e^{-m}} > 0 \ (j \neq t) \quad \Longrightarrow \quad \sum_k g_k = \frac{e^{-m}}{1 + e^{-m}} > 0$$

기호	읽는 법	느낌
g_t	지 티	정답 칸 gradient. 반올림으로 **0**
g_j	지 제이	오답 칸 gradient. **살아 있음**
$\sum_k g_k$	합	0이어야 하는데 **양수**가 됨

아주 작은 숫자 예 — 실측

$z = [9, -9, -9]$, 정답 $t = 0$. 이때 $m = 17.31$(절벽 16.64 초과).

구현	정답칸 gradient	오답칸 (각)	합
표준 (프레임워크)	0.0	+1.523e-8	+3.046e-8
고친 구현	-3.046e-8	+1.523e-8	0 (구조적으로)
이론값 e^{-m}	-3.046e-8	+1.523e-8	0

숫자가 전부 실제 계산 결과입니다. 표준 구현은 **정답 칸만 0**, 그래서 합이 3.05×10^{-8}입니다. 절대값은 작아 보이지만 STEP 7에서 이 작은 값이 **증폭**됩니다.

핵심

버그는 "gradient가 작다"가 아니라 **"한 칸만 0이 되어 합이 0이 아니다"**입니다. 작은 것은 정상이고, **틀린 방향**이 문제입니다.

확인 문제 6-1

표준 구현에서 두 오답 gradient가 각각 1.523×10^{-8}이면 합은 얼마이며, 왜 0이 아닌가?

Adam은 "작은 값"과 "0"을 전혀 다르게 다룬다

> **왜 필요한가.** 여기까지는 "아주 작은 오차"였습니다. 그런데 Adam 계열 옵티마이저는 gradient를 **자기 크기로 나눠서** 방향만 남깁니다. 그래서 0이 되면 정보가 완전히 사라지고, 다시 살아나면 **과거의 작음**이 사라진 채 큰 스텝이 나옵니다. 이것이 Slingshot입니다.

비유

자동 볼륨 조절 마이크. 아주 작은 소리도 크게 증폭합니다. 그런데 마이크가 꺼져 있다(0)가 다시 켜지면, 증폭기는 **지난 몇 초의 조용함을 기억**한 채라서 갑자기 굉음이 터집니다. Adam의 v(제곱평균)가 그 기억입니다.

수식

$$\theta \leftarrow \theta - \eta \left(\frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon} + \lambda \theta \right)$$

기호	읽는 법	느낌
$\hat{m}$	엠 햇	gradient의 이동평균 (방향)
$\sqrt{\hat{v}}$	루트 브이 햇	gradient 크기의 이동평균. **나누는 값**
ϵ	엡실론	0으로 나누지 않게 하는 작은 상수. 기본 10^{-8}
λ	람다	weight decay 세기

핵심은 $\hat{m}/\sqrt{\hat{v}}$ 가 **크기 정보를 지운다**는 점입니다. gradient가 10^{-3}이든 10^{-8}이든 비율은 1에 가깝습니다. 그래서 **0이 되는 순간만 정보가 끊깁니다.**

아주 작은 숫자 예 — 실측

$\epsilon = 10^{-8}$일 때 $|g|/(|g| + \epsilon)$:

|g|	스텝 비율	뜻
1e−2	1.000	평소. 최대 속도
3e−8	0.750	아직 절반 이상
1e−8	0.500	반으로 줄어듦
1e−13	0.00001	사실상 정지

이제 Slingshot의 순서를 따라가 봅시다.

#	일어나는 일
1	모델이 학습 데이터를 외워 margin이 커지고, **절벽을 넘는다**
2	정답 gradient가 0 → $\hat{v}$가 **지수적으로 0으로 감소**, weight decay는 파라미터를 조금씩 줄임
3	파라미터가 줄어 어떤 예제가 **다시 절벽 아래로** 내려옴
4	그 예제의 gradient가 **되살아남**. 그런데 $\sqrt{\hat{v}}$는 죽어 있어서 큰 스텝
5	손실이 **튄다**. $\hat{v}$가 리셋되고 2번으로 돌아감 → 주기적 스파이크

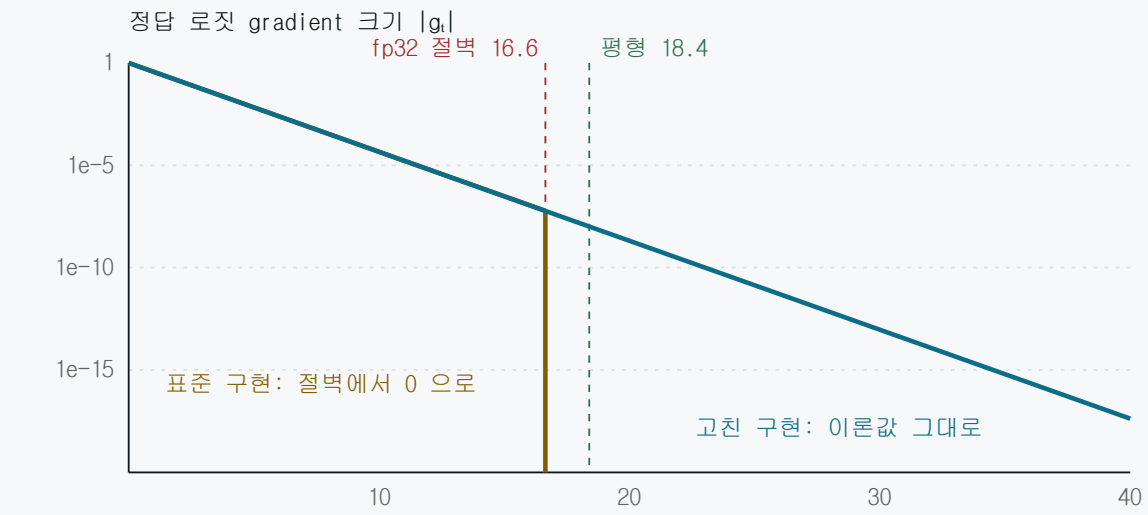


그림 2. 로그 눈금이라 직선은 지수 감쇠를 뜻한다. 표준 구현은 $c_{32} = 16.6$에서 0으로 떨어지고, 고친 구현은 exp 언더플로($m \approx 88$)까지 따라간다.

> 확인 문제 7-1
>
> $|g| = 10^{-13}$, $\epsilon = 10^{-8}$일 때 스텝 비율을 계산하라. Slingshot에서 이 상태가 왜 위험한가?

계산 메모 – 손으로 검산해 보세요

STEP 08

수정 1 — 1을 더하지 않는다

> **왜 필요한가.** 증상을 봉합하지 않고 **원인을 없애는** 방법입니다. 원인은 "가장 큰 항이 정확히 1이라는 사실을 분모에 더하는 것"이었습니다. 그러면 그 항을 **아예 합에서 빼면** 됩니다.

> 비유
>
> 무게를 잴 때 빈 접시 무게를 영점 조정해 둡니다. 접시 위에 아무것도 없을 때 **"접시 무게 + 0"을 다시 계산하지 않습니다.** 영점이 이미 0이니까요. 표준 구현은 매번 "1 + 아주 작은 수"를 다시 계산하다가 그 작은 수를 잃어버립니다.

수식

가장 큰 자리(또는 정답 자리)를 상수 0으로 고정하고, 그 자리를 합에서 제외합니다. 그리고 로그 쪽은 $1 + x$에 강한 **softplus**를 씁니다.

$$\tilde{z}_k = \mathbf{1}[k \neq i_{\max}]\,(z_k - z_{i_{\max}}), \qquad m = -\,\mathrm{LSE}_{j \neq i_{\max}}\, \tilde{z}_j$$

$$\log \mathrm{softmax}(\mathbf{z})_k = \tilde{z}_k - \mathrm{softplus}(-m), \qquad \mathrm{softplus}(x) = \log(1 + e^{x})$$

기호	읽는 법	느낌
$i_{\max}$	아이 맥스	가장 큰 로짓의 위치
LSE	로그-섬-이엑스피	$\log \sum e^{(\cdot)}$. 넘치지 않게 계산하는 표준 함수
$\mathrm{softplus}$	소프트플러스	x가 아주 작을 때도 $\log(1 + e^{x})$를 **정확히** 계산

아주 작은 숫자 예

계산	표준	고친 구현
손실 (m=17.31)	0.0	3.046e−8
정답 log-확률	0.0	−3.046e−8
gradient 합	+3.046e−8	0

numpy 로 옮긴 구현입니다(실행·검증 완료).

```
import numpy as np

def softplus(x):
    return np.maximum(x, 0.0) + np.log1p(np.exp(-np.abs(x)))

def log_softmax(z):
    i_max   = z.argmax(-1, keepdims=True)
    z_max   = np.take_along_axis(z, i_max, axis=-1)
    is_max  = (np.arange(z.shape[-1]) == i_max)
    shifted = np.where(is_max, 0.0, z - z_max)     # argmax 자리는 상수 0
    others  = np.where(is_max, -np.inf, shifted)   # 그 항을 합에서 제외
    m = -np.log(np.sum(np.exp(others), axis=-1, keepdims=True))
    return shifted - softplus(-m)

def cross_entropy(z, t):
    z_t    = np.take_along_axis(z, np.array([t]), axis=-1)
    is_t   = (np.arange(z.shape[-1]) == t)
    others = np.where(is_t, -np.inf, z - z_t)      # target 을 합에서 제외
    m = -np.log(np.sum(np.exp(others), axis=-1))
    return softplus(-m)
```

함정 — 여기서 많이 틀립니다
고친 구현에서도 $q = \exp(\log_\mathrm{softmax})$**를 만들어** $q_t - 1$**을 하면 다시** 0입니다. 실측: fp32 에서 $q_t = 1.0$이 되어 $q_t - 1 = 0$. 그래서 이 수정은 "확률을 잘 계산하는 것"이 아니라 **정답 로짓이** $K - 1$**개의 차이로만 흐르게 만드는 구조**입니다. autodiff가 $g_t = -\sum_{j \neq t} g_j$를 자동으로 만들어 줍니다. 프레임워크의 자동미분을 반드시 그대로 쓰세요.

확인 문제 8-1

고친 구현에서 $\sum_k g_k = 0$이 **구조적으로** 보장되는 이유를 한 줄로 설명하라.

STEP 09

버그 2 — 증류에서는 두 softmax를 빼면 안 된다

> **왜 필요한가.** 앞의 버그는 "정답을 너무 확신할 때" 생겼습니다. 두 번째는 정반대 상황, **학생이 이미 교사와 똑같을 때** 생깁니다. 증류(knowledge distillation)를 쓰는 사람에게는 이쪽이 더 직접적입니다.

비유

두 사람의 키를 각각 cm 단위로 반올림해서 적은 뒤 뺍니다. 같은 사람인데도 **1cm 차이**가 남습니다. 차이를 먼저 재면 0입니다. **각각 재서 빼는 것**과 **차이를 직접 재는 것**은 다릅니다.

수식

증류 손실은 교사의 확률분포 p에 대한 cross-entropy입니다. gradient는

$$g_k = q_k - p_k, \qquad q = \mathrm{softmax}(z_S), \quad p = \mathrm{softmax}(z_T)$$

$z_S = z_T$이면 $q = p$이므로 $g = 0$이어야 합니다. 실제로는 두 가지 이유로 0이 아닙니다.

#	원인
1	1에 가까운 두 확률을 **따로 저장한 뒤 빼면** 유효자리가 사라진다 (STEP 4의 눈금)
2	forward는 `softmax`, backward는 `log_softmax` 경로라 **연산 순서가 미세하게 다르다**

아주 작은 숫자 예 — 실측

$z_T = [2, 1, 0]$, $z_S = z_T + [-10^{-7}, 10^{-7}, 0]$ (fp32).

구현	g_0	g_1	g_2	합
표준 (따로 계산해 뺌)	−5.96e−8	+1.45e−8	−1.12e−9	−4.62e−8
고친 구현	−4.60e−8	+4.14e−8	+4.51e−9	0
정확한 값 (fp64)	−4.60e−8	+4.14e−8	+4.51e−9	0

표준 구현은 둘째 칸을 **3배 작게**, 셋째 칸은 **부호를 뒤집어** 계산하고 합도 0이 아닙니다. Adam은 이 잔차가 ϵ보다 크면 **크기와 무관하게 η만큼** 밀어붙이므로, bf16(잔차 $\sim 10^{-3}$)에서는 학생이 **표류**합니다(논문 Figure 7).

확인문제 9-1

표준 구현 gradient 세 개를 더해 보라. 0이 되어야 하는데 얼마인가?

계산 메모 – 손으로 검산해 보세요

STEP 10

수정 2 — expm1과 log1p로 "차이"를 직접 잰다

> **왜 필요한가.** STEP 9의 처방은 자연스럽습니다. 확률로 바꿨다가 빼지 말고, **로짓 차이** $\delta = z_S - z_T$에서 바로 gradient를 만듭니다. 그런데 $e^x - 1$과 $\log(1+x)$를 그냥 계산하면 같은 문제가 반복됩니다. 그래서 전용 함수가 있습니다.

비유

몸무게 변화를 재고 싶습니다. 방법 A는 "이번 달 몸무게 - 지난달 몸무게"를 각각 재서 뺍니다. 방법 B는 **변화량만 재는 저울**을 씁니다. 방법 B가 $expm1(e^x - 1)$이고, $log1p$는 그 반대 방향의 전용 저울입니다.

수식

$\delta = z_S - z_T$라 하고, p로 중심을 옮긴 뒤 로그비율 s, 차이 u를 만듭니다.

$$d_k = \delta_k - \sum_i p_i \delta_i, \qquad s_k = d_k - \log 1\mathrm{p}\left(\sum_i p_i \,\mathrm{expm}\,1(d_i)\right), \qquad u_k = p_k \,\mathrm{expm}\,1(s_k)$$

$$g_k = u_k - q_k \sum_i u_i$$

기호	읽는 법	느낌
δ	델타	학생 로짓 − 교사 로짓. **차이를 먼저** 만든다
$\mathrm{expm}\,1$ / $\log 1\mathrm{p}$	이엑스피 마이너스 원 / 로그 원 플러스	$e^x - 1$ 과 $\log(1+x)$. x가 10^{-8} 이어도 정확
s_k	에스 케이	$\log(q_k/p_k)$. 1을 더하지 않고 비율을 직접
g_k	지 케이	$u_k - q_k \sum_i u_i$. 남은 오차까지 **합 0**으로 투영

아주 작은 숫자 예

같은 예($z_T = [2, 1, 0]$, $\delta = [-10^{-7}, 10^{-7}, 0]$)에서 결과가 **정확한 값과 자리까지 일치**합니다.

구현	g_0	g_1	g_2
표준	−5.96e−8	+1.45e−8	−1.12e−9
고친 구현	−4.5955e−8	+4.1442e−8	+4.5131e−9

논문의 JAX 구현은 forward 손실은 그대로 두고 `stop_gradient`로 **backward에만** 이 gradient를 넣습니다 — 손실값은 같고 **미분만 교체**합니다.

확인 문제 10-1

$x = 10^{-8}$일 때 $\log(1+x)$를 그냥 계산하면 왜 0이 되는가? $log1p$는 왜 괜찮은가?

STEP 11

왜 기본값 $\epsilon = 10^{-8}$이 최악인가 — 그리고 정리

> **왜 필요한가.** 이 논문에서 가장 예쁜 부분입니다. "왜 지금까지 산발적으로만 관찰됐는가"에 대한 답이 **숫자 하나**로 나옵니다. AdamW가 멈추는 지점을 계산하면 절벽 바로 위에 걸립니다.

비유

고무줄(weight decay)이 파라미터를 당기고, 모터(Adam 스텝)가 반대로 밉니다. 모터는 입력이 ϵ보다 작아지면 **힘이 급격히 죽습니다.** 그래서 둘이 만나는 지점이 정해집니다. 그 지점이 하필 **벽(절벽) 바로 뒤**에 있습니다.

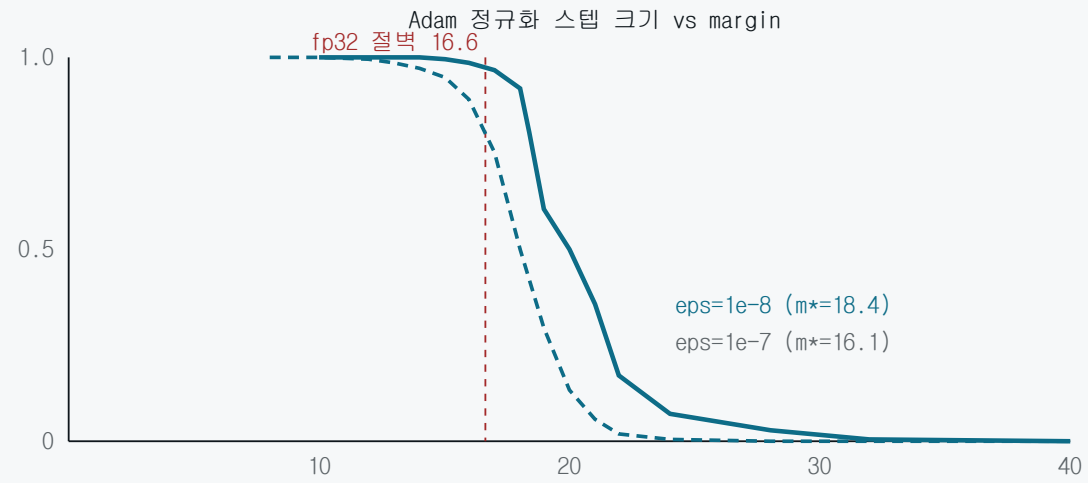


그림 3. $y = 1/(1 + e^{m - \log(1/\epsilon)})$. 스텝은 $\log(1/\epsilon)$까지 1을 유지하다 급격히 죽는다. 실선이 기본값, 점선이 $\epsilon = 10^{-7}$.

수식

파라미터가 멈추는 정류점에서 $\hat{m} = g$, $\sqrt{\hat{v}} = |g|$이므로

$$\frac{g}{|g| + \epsilon} + \lambda\theta = 0 \implies |g| \approx \epsilon\lambda|\theta| \implies m^\star \approx \log\frac{1}{\epsilon}$$

기호	읽는 법	느낌
$m^\star$	엠 스타	학습이 스스로 멈추는 margin
ϵ	엡실론	Adam 기본값 10^{-8}. 이 값이 천장을 정한다

정확히는 상수항 $\log(1/(\lambda|\theta|))$이 더 붙지만, 실측에서는 $\log(1/\epsilon)$ 항이 지배합니다(아래 표).

아주 작은 숫자 예

ϵ	예측 $m^\star$	논문 실측	판정
1e−8	18.42	17.9	**절벽(16.64)보다 위 → 위험**
1e−12	27.63	27.4	절벽보다 훨씬 위
1e−16	36.84	36.8	fp64 절벽(36.74)도 넘음
1e−7	16.12	–	**절벽 아래 → 안전**

기본값 10^{-8}은 $18.42 > 16.64$라서 **절벽을 넘은 뒤에야 평형에 도달**합니다. $\epsilon \geq 10^{-7}$이면 $16.12 < 16.64$이므로 표준 구현도 절벽에 닿기 전에 멈춥니다.

> 세 줄 정리
> ① 정답을 확신할수록 $1 + e^{-m}$의 작은 항이 반올림으로 사라지고, **정답 gradient만 0**이 되어 합이 0이 아니게 된다. ② Adam은 그 비대칭을 증폭해 **주기적 손실 스파이크(Slingshot)**를 만들고, 증류에서는 **학생 표류**를 만든다. ③ 1을 더하지 않는 구현(softplus·argmax 제외)과 차이를 먼저 재는 구현(expm1·log1p)으로 **구조적으로** 고친다.

> 확인 문제 11-1
> 내 학습 스크립트에서 오늘 확인할 수 있는 것 세 가지를 적어 보라.

계산 메모 – 손으로 검산해 보세요

스스로 확인하기

1

로짓 $z = [0, 0, 0]$일 때 softmax 확률은?

2

$z_t = 5$, 나머지 두 로짓이 $0, 0$일 때 margin m은?

3

$z = [0, 0]$ (2클래스)에서 정답 $t = 0$일 때 gradient 두 개를 구하라.

4

$m = 15$일 때 $1 - q_t$를 구하라. 이 값이 fp32에서 표현되는가(반올림 경계 5.96×10^{-8})?

5

fp32에서 정답 gradient가 0으로 반올림되기 시작하는 m을 $p \ln 2$로 계산하라.

6

$|g| = 3 \times 10^{-8}$, $\epsilon = 10^{-8}$일 때 Adam의 스텝 비율은?

7

고친 구현에서 정답 gradient가 -3.046×10^{-8}이고 오답이 두 개다. 각 오답 gradient는?

8

$\epsilon = 10^{-7}$일 때 $m^\star = \log(1/\epsilon)$을 계산하고, fp32 절벽(16.64)과 비교해 안전한지 판정하라.

9

(서술) StableMax는 $\exp$ 대신 $x + 1$ 꼴의 선형 함수를 쓴다. 왜 이것이 margin을 10^3까지 폭발시킬 수 있는가?

10

(서술) 이 논문이 **대규모 LLM 학습에서도** 문제라고 주장하지 않는 이유를 한 줄로.

계산 과정까지

1

$e^0 = 1$씩이라 합이 3, 각각 $1/3 = 0.333$. **로짓이 모두 같으면 균등분포.**

2

$m = 5 - \log(e^0 + e^0) = 5 - \ln 2 = 5 - 0.693 = \mathbf{4.307}$.

3

$q = [0.5, 0.5]$. gradient는 $q - \mathbf{1} = [-0.5,\ +0.5]$. 합 $= 0$.

4

$1 - q_t = e^{-15}/(1 + e^{-15}) = \mathbf{3.06 \times 10^{-7}}$. 반올림 경계 5.96×10^{-8}보다 **크므로 표현됨**. 즉 $m = 15$는 아직 안전 구간.

5

$24 \times \ln 2 = 24 \times 0.693 = \mathbf{16.64}$. 실측으로 $m = 17$에서 `1+e^-17`이 정확히 1.0이 된다.

6

$3 \times 10^{-8}/(3 \times 10^{-8} + 10^{-8}) = 0.75$. 최대 속도의 **75%**로 여전히 밀어붙인다.

7

합이 0이어야 하므로 각각 $+1.523 \times 10^{-8}$. ($3.046/2 = 1.523$)

8

$\log(10^7) = 7 \ln 10 = 7 \times 2.3026 = \mathbf{16.12}$. $16.12 < 16.64$이라 **절벽에 닿기 전에 멈춘다 — 안전**.

9

exp는 margin이 커지면 확률이 1에 **포화**되어 gradient가 지수적으로 0이 된다. 반대로 선형 함수는 확률이 1에 도달하지 않으므로 gradient가 **죽지 않고**, 모델이 계속 margin을 키운다. 그래서 $10^3 \sim 10^4$까지 폭발한다(논문 Table 1).

10

대규모 학습에서는 logit margin이 보통 한 자리수라 절벽(특히 fp32의 16.6)에 닿지 않고, 저자도 conclusion에서 "우리 실험의 극단적 margin은 표준 대규모 학습에서 흔하지 않다"고 밝힌다. 즉 이 논문이 검증한 것은 **grokking 규모의 실험**이다.

더 공부하려면

원문과 배경

자료	무엇을 볼까
원문 arXiv:2610.31642	§2(절벽), §5(평형), Figure 2(Slingshot)만 먼저. 8쪽이라 30분
Thilak et al. 2022	Slingshot 현상의 원 관찰. 이 논문의 출발점
Prieto et al. 2025 (StableMax)	비교군. 왜 선형 근사가 margin을 폭발시키는지
Liu et al. 2026	zero-sum projection. 왜 절반만 고치는지
Nanda 2023 / Varma 2023	grokking 실험 세팅의 표준

내 프로젝트에 적용하기 (CV 앱)

상황	할 일
bf16으로 학습 중	절벽이 **5.5**로 매우 낮다. loss가 0에 붙거나 급등하면 margin을 로깅해서 확인
AdamW 기본값 사용	`eps=1e-7`로 올린다. 한 줄, 부작용 없음, $m^\star = 16.1$로 절벽 아래
증류를 쓴다	고친 soft-target cross-entropy를 쓴다. 특히 student를 teacher weight로 초기화하거나 T가 클 때
단일 분류기, fp32, label smoothing	우선순위 낮음. 지금 동작하면 그대로 둔다

> 다음 단계 제안
>
> ① 내 학습 스크립트에 margin 로깅 한 줄 추가 `m = logits.max(1).values - ...` ② AdamW `eps=1e-7`로 바꾸고 손실 곡선 비교 ③ 증류를 쓴다면 고친 soft-target loss로 A/B.

이 워크북의 모든 수치는 로컬에서 실제 계산한 값이며, 코드 조각은 실행·컴파일 검증을 마쳤습니다. 원문은 arXiv:2610.31642v1 (2026-10-22) HTML 전문을 읽고 정리했습니다.